LUL
Emiru playing Honkai: Star Rail
170,613 views

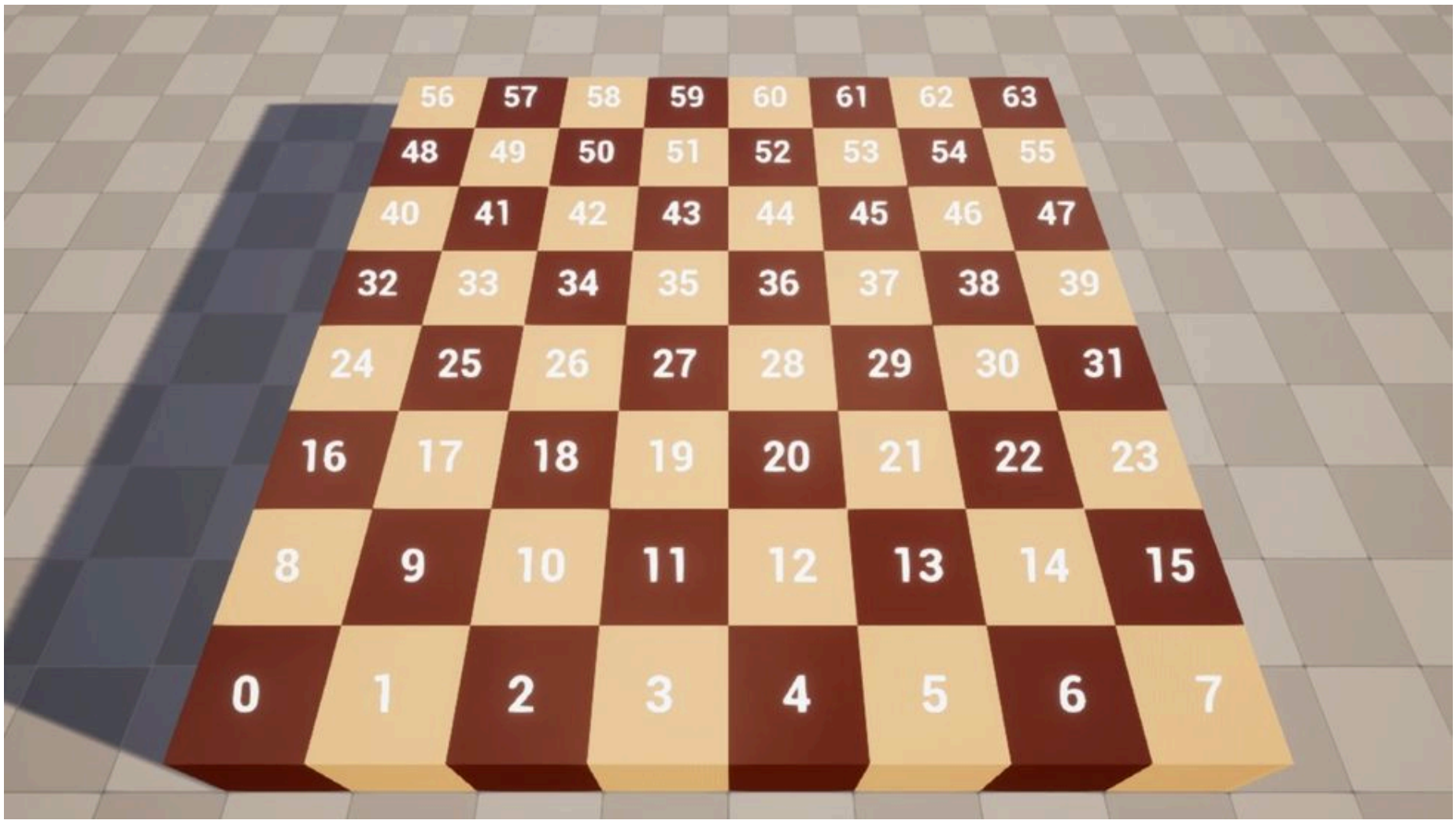
22

After I saw it, I thought to myself, If chess would make me feel like 1/10 of what she was feeling, I probably could play lots of chess games and get better at it. And then suddenly, I got hit by a revolutionary idea. If we look back at that Twitch clip, we can see that she was playing a game named Honkai Star Rail. And if we Google it, we can see that it's a gacha game.

If you don't know what a gacha game is, it's a game that has its own in-game currency, and the game encourages you to use that currency for in-game gambling.

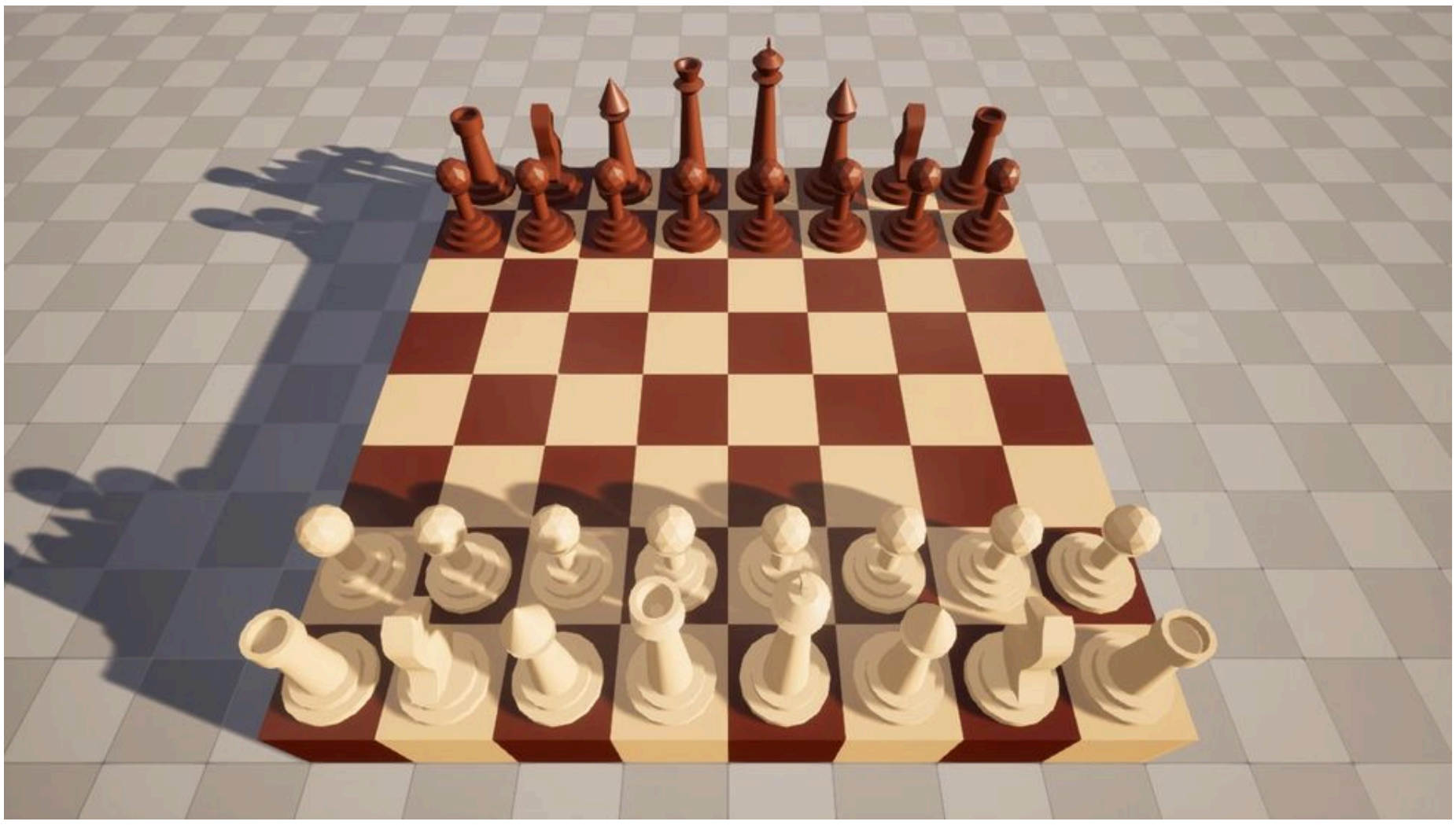
Well, maybe it's not straight-up gambling. I did some research on gacha mechanics and found out that some mechanics are called surprise mechanics, which are ethical and fun. At least that's what multi-billion-dollar companies claim. And I don't see any reason why they would lie to us (they might lie because of easy money).

So what if I took gacha game surprise mechanics and added them to a chess game? Can a gacha chess game make me a better chess player? Can I even make a normal chess game? I think to find answers to all these questions, I should start by creating a chess game. Let's start by creating a chessboard.



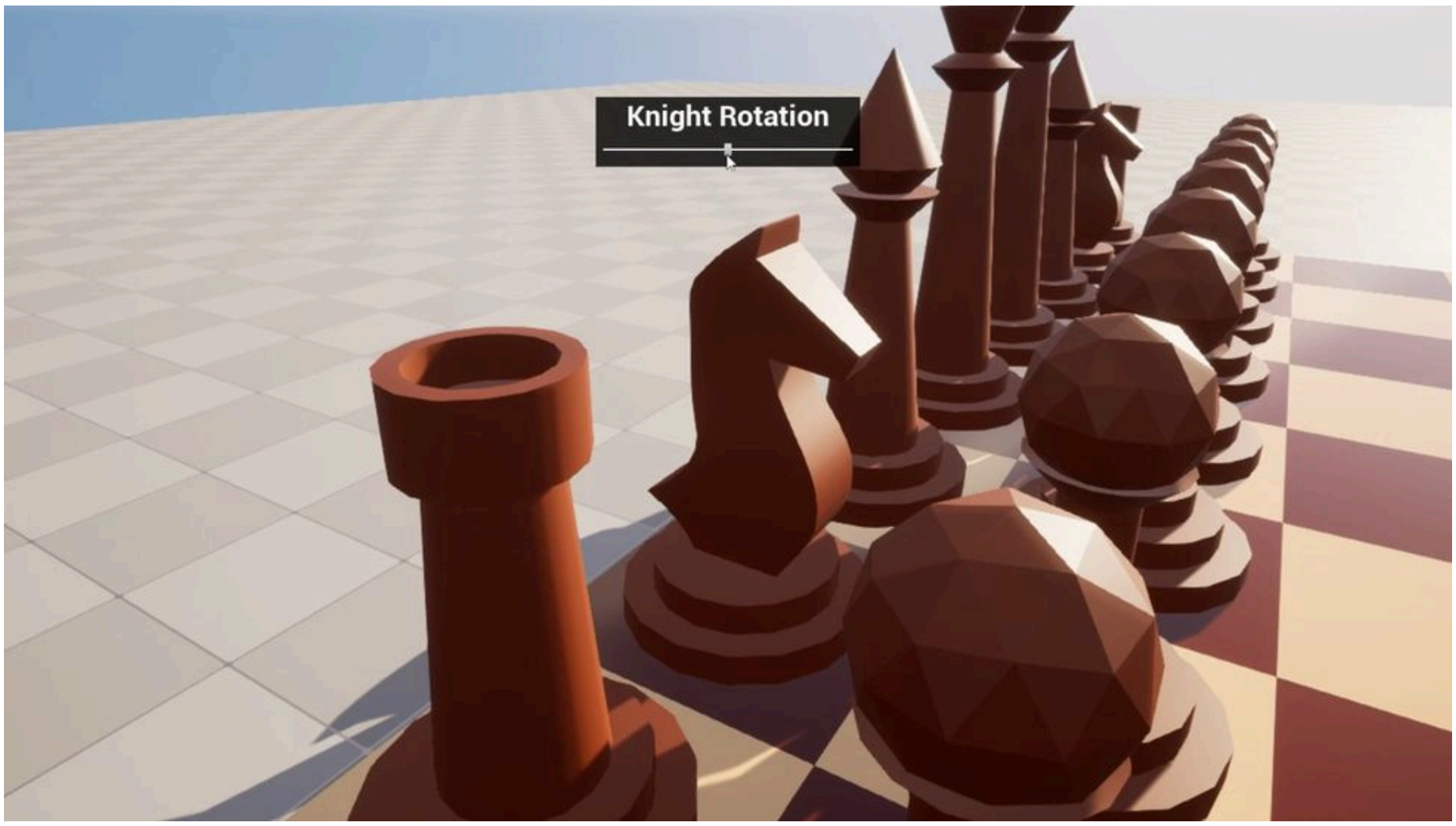
To create it, I set 64 cubes in an 8 by 8 grid layout and set their colors to something that's not too harsh for my eyes. Each of those cubes has its own unique index number assigned to it, so whenever any of those cubes is clicked, we will know which square on the chessboard was clicked by looking at the cube's index value. But before we can utilize those index values, we need to put some pieces on the chessboard.

So I modeled a temporary set of chess pieces in the blender, imported them into the unreal engine, and then set them up on the chessboard.



Then I had a look at the chess pieces, and I had this question: Does it matter in which direction the knight pieces are facing?

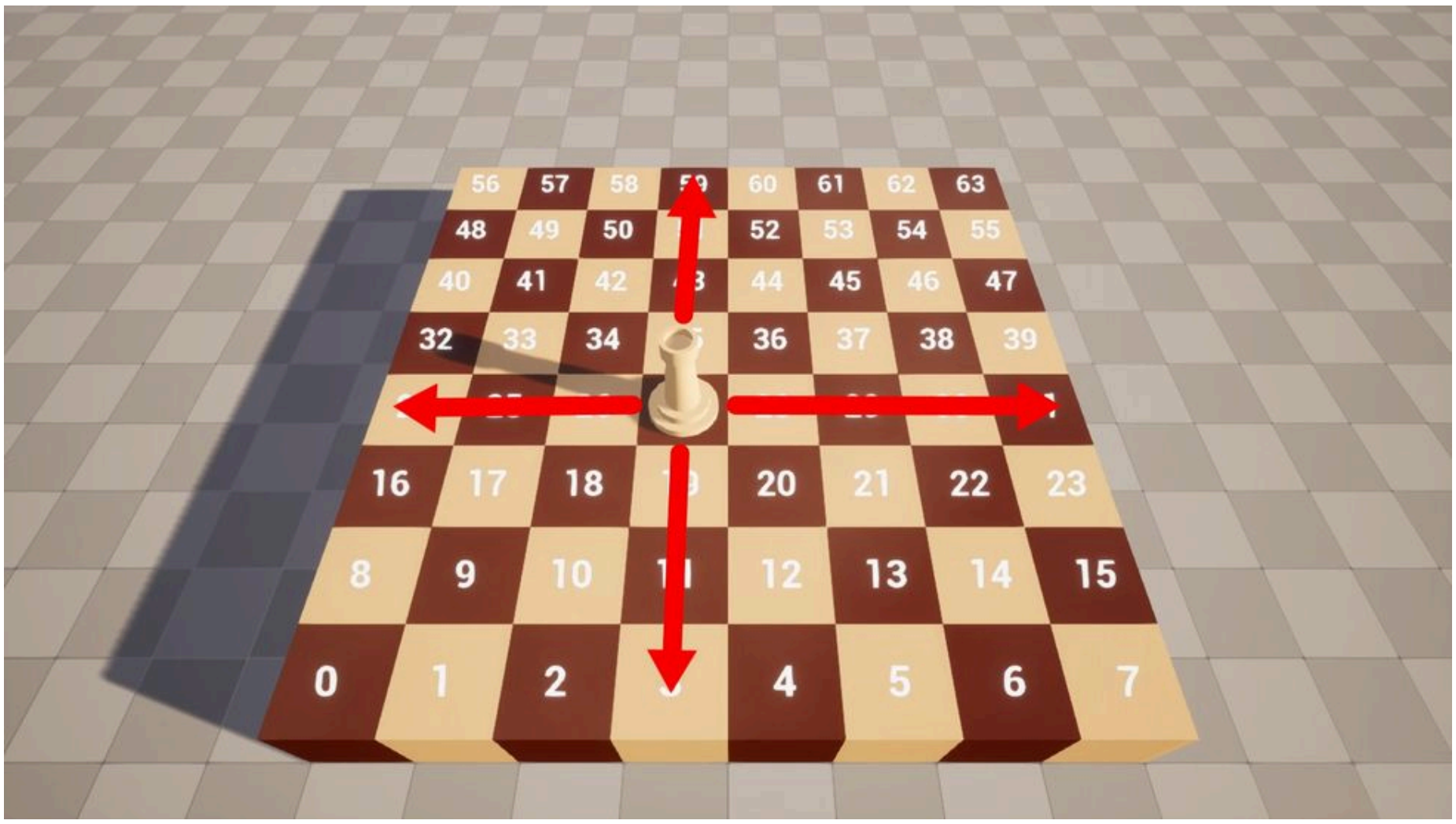
I did some quick research, and it seems that it's a personal preference thing. So I made them rotate around their axis, so it would match everyone's preference at least once per rotation. But in all seriousness, I probably should make an option in the game settings where I can change the direction of the knight pieces.



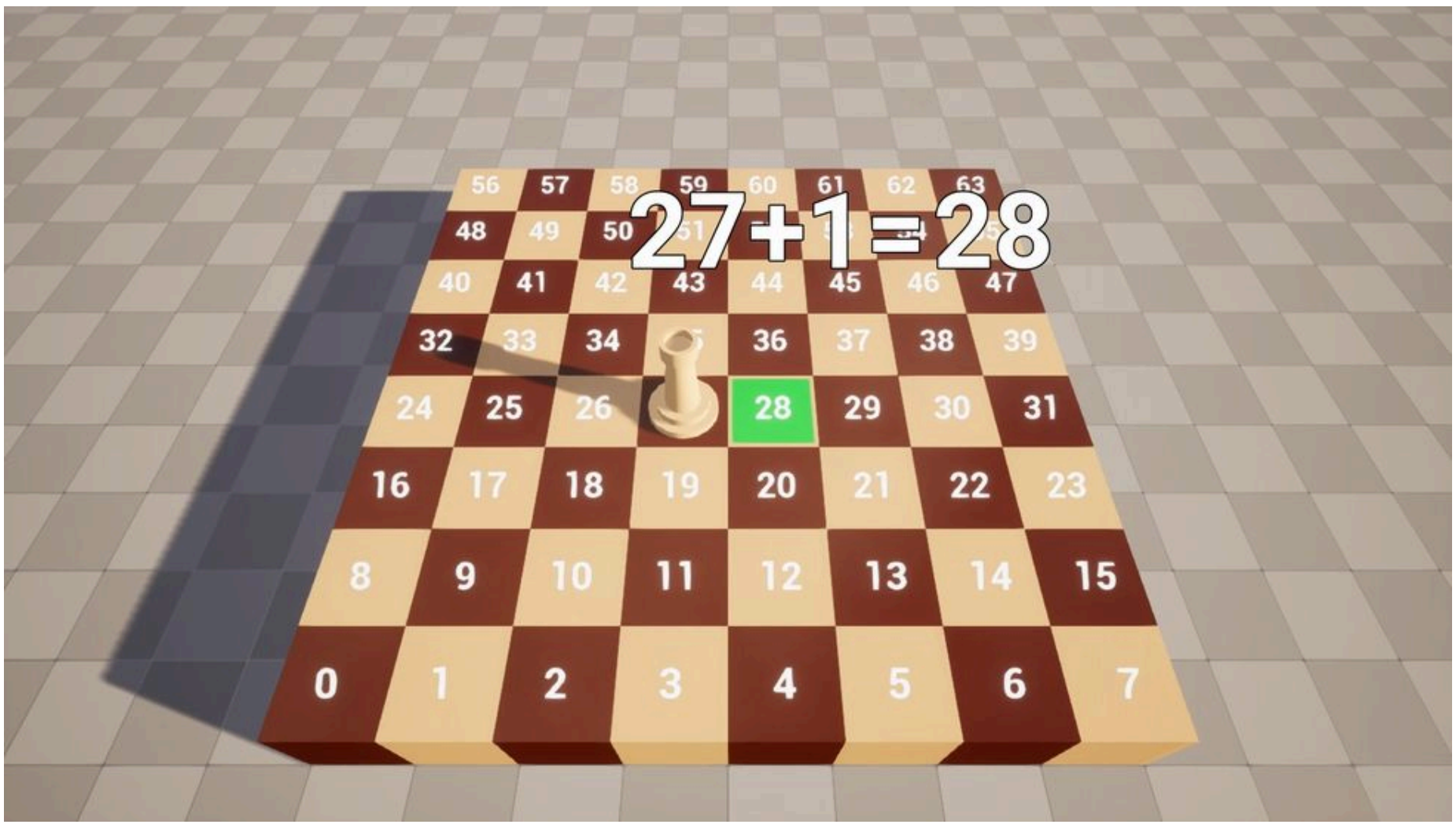
Alright, enough about the knight pieces, let's return to chess gameplay development.

So I added the ability to move pieces around and the ability to capture pieces. Theoretically, this is enough to be able to play a game of chess. But in reality, I can move any piece to any square, and I can capture any piece with any piece, and there's no move order, so I can go with the same color pieces several times in a row. Because I'm really bad at chess, I don't trust myself that I will be able to play the correct game of chess if I need to enforce the rules of chess by myself. So I guess it's time to implement a movement logic for each piece.

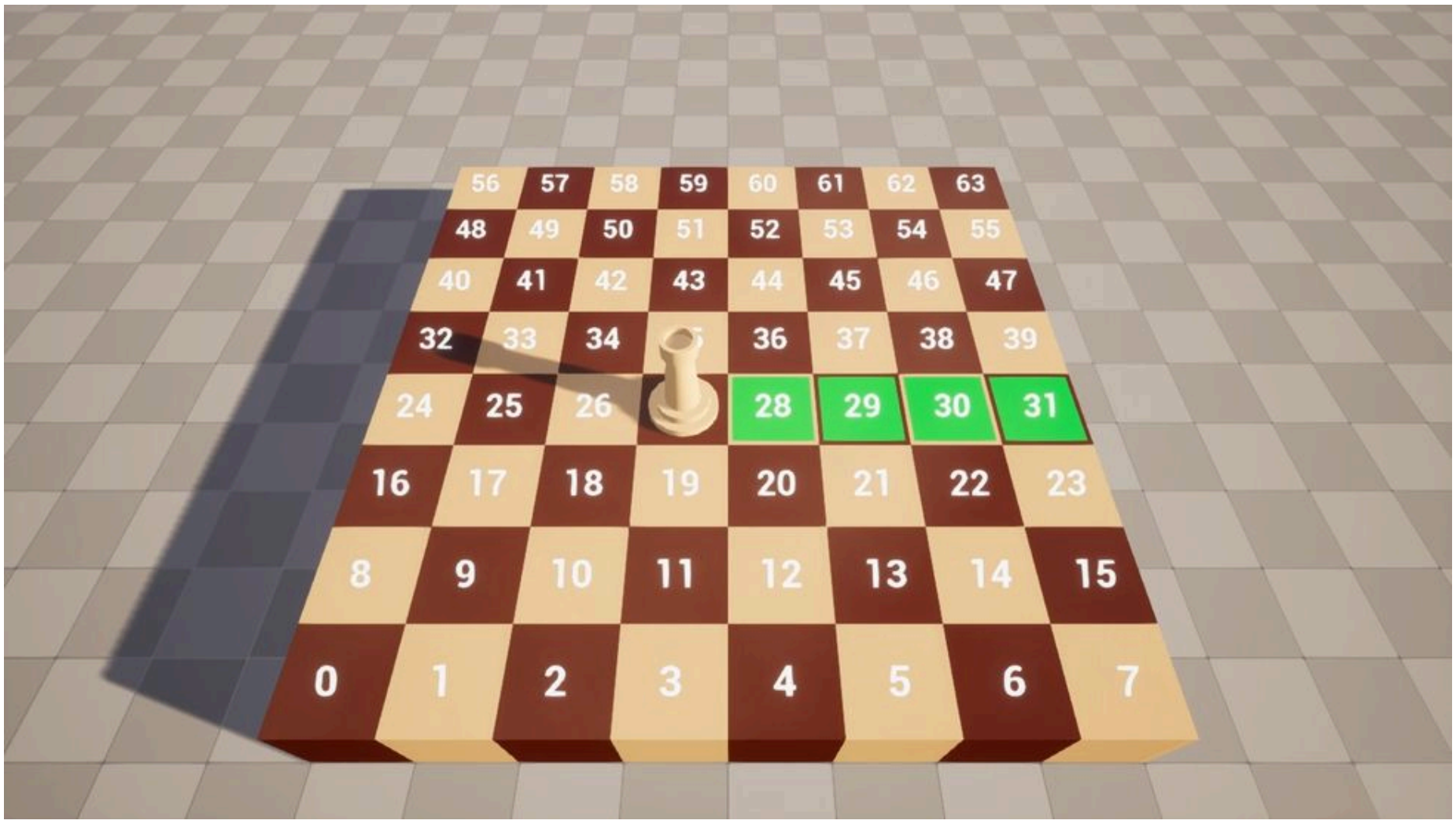
Let's start with the movement of rooks. We can see that our current rook piece is placed on the square with an index of 27 and we know that the rook can move forward, backward or sideways.



So how would we find all the valid movements on the right side of the rook? If we take our piece's current index and add one to it, we will see that we get the index of the square to the right of our rook, which is 28.

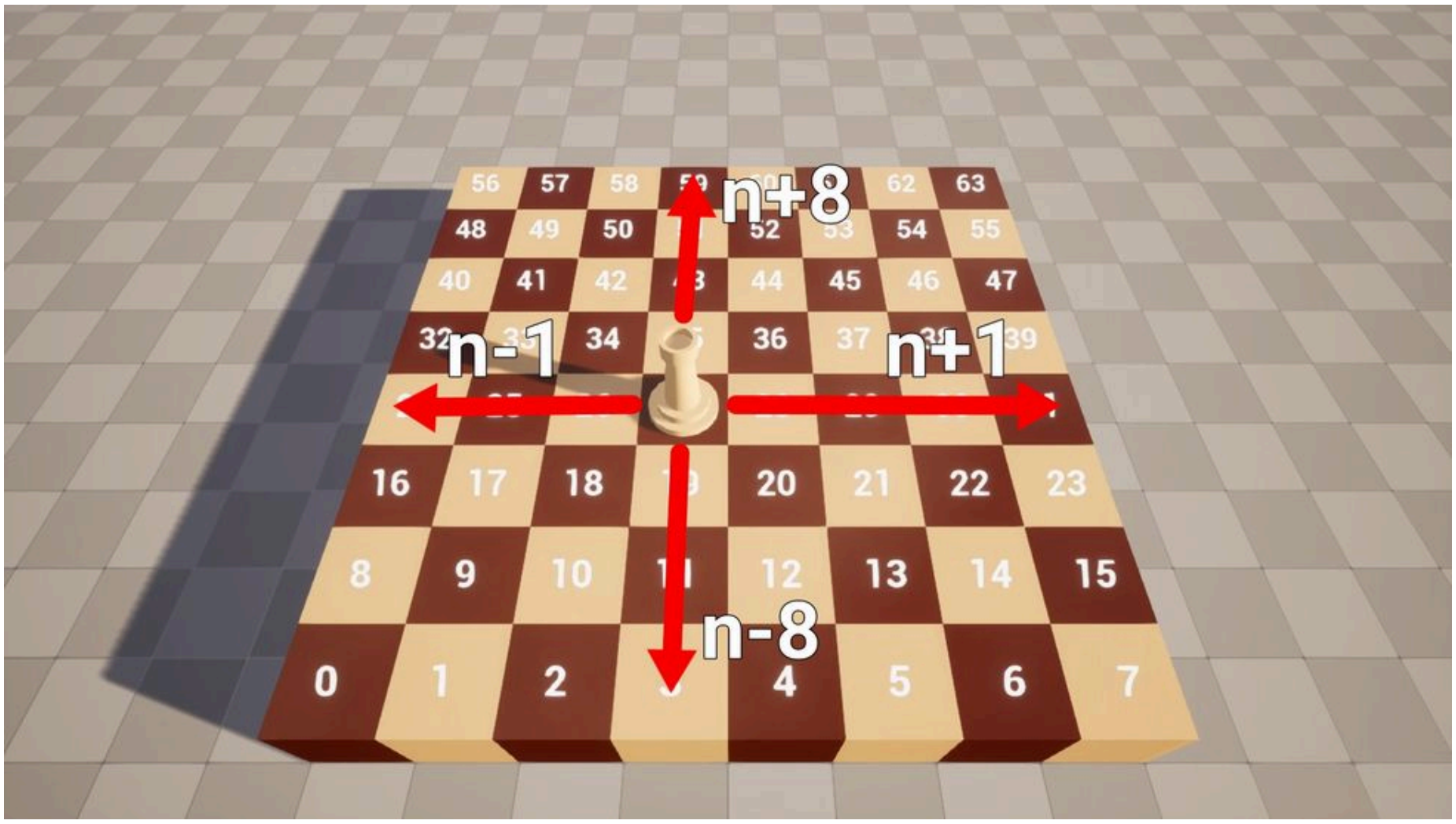


If we take the right square index and add one to it again, we will find the next valid move on the right side, which is 29. So now, if we repeat the same steps until we reach the edge of the chessboard or find a square with another piece on it, we should find all the valid moves on the right side of the rook.

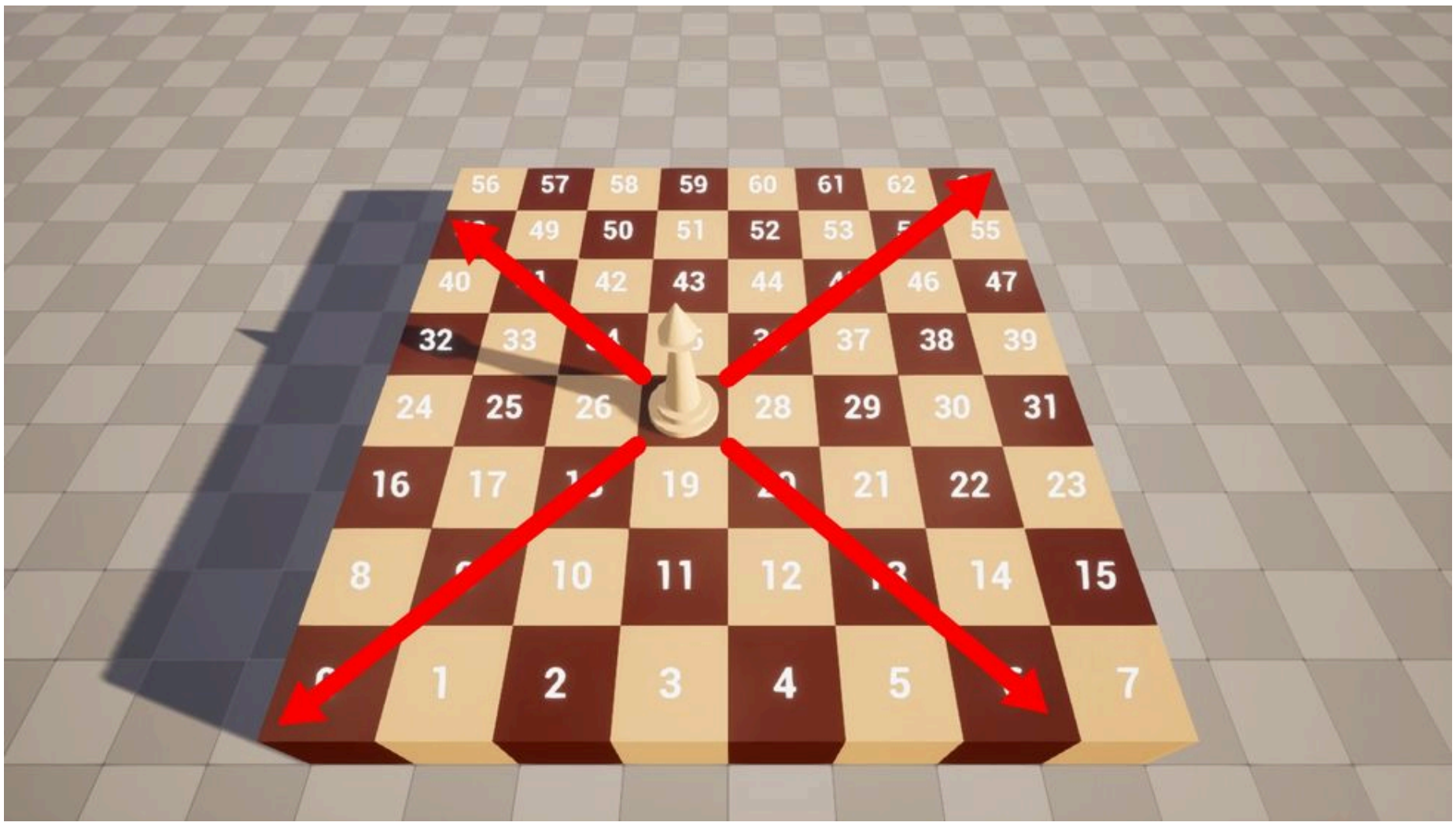


To find all the valid moves on the left side, we repeat the same steps as for the right side, but instead of adding one to the index values, we subtract one from them, and we will get all the valid moves for the left side.

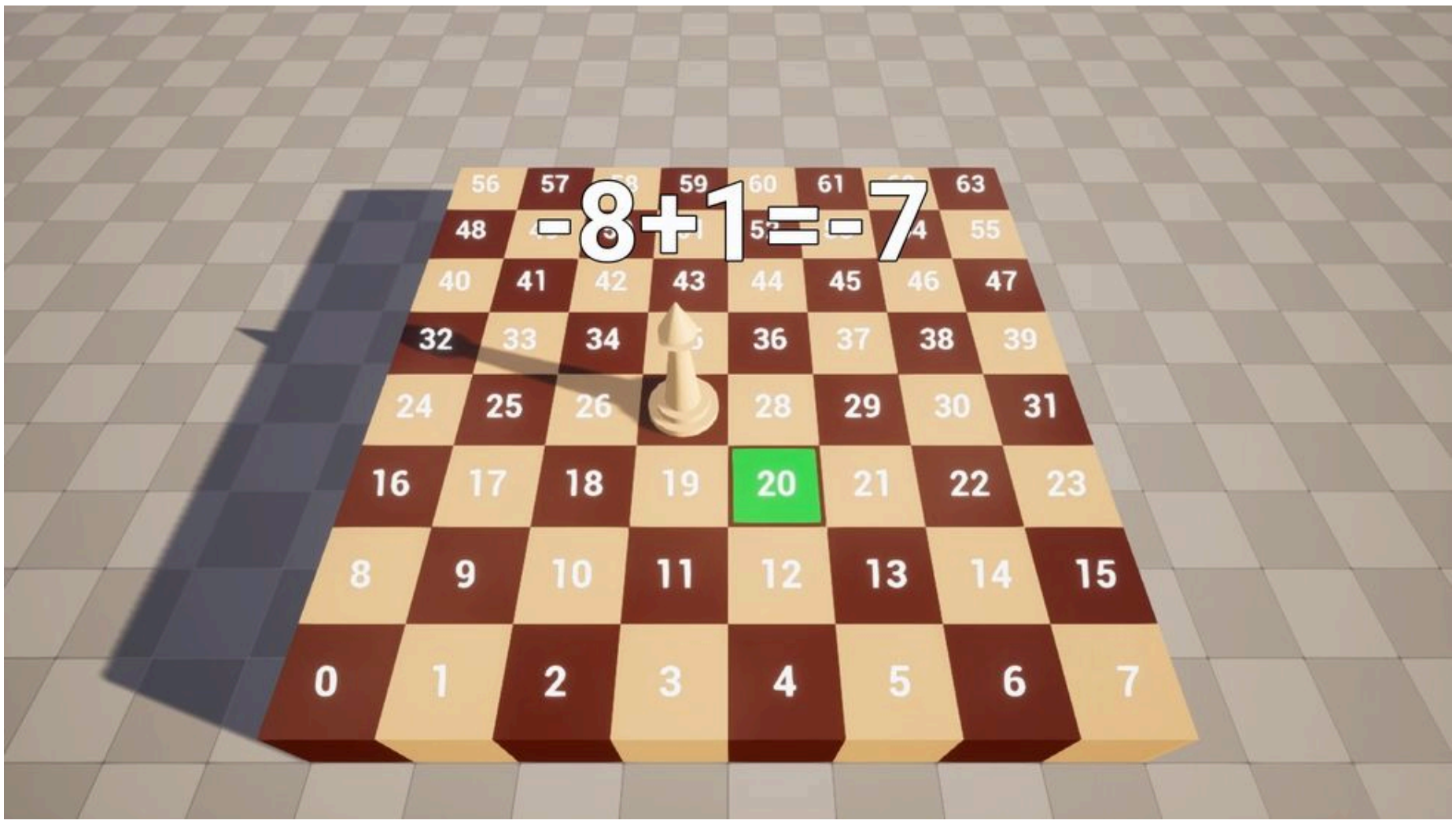
Now that we know how to get valid moves for the left and right sides, how do we find the valid moves for the front and back sides? We can use the same algorithm we used before for the left and right sides, but instead of adding or subtracting index values by one, we will use a different number. To get the new number, we can look at the chessboard and see that the index of the square below our rook is 19. If we subtract 19 from the rook index 27, we will see that the difference is 8. Or if we look at the width of the chessboard, we see that it's 8 squares, so it makes sense that by adding or removing the width of the board, we will get an index of a forward or backward square.



So now we know how to find forward, backward, and side square indexes. That should be enough to find valid movements for rooks, so let's have a look at the movements of a bishop. The bishop moves only on diagonals. To find the diagonal indexes, we can use the same algorithm we used for the rook; we just need to add or subtract different numbers from indexes.

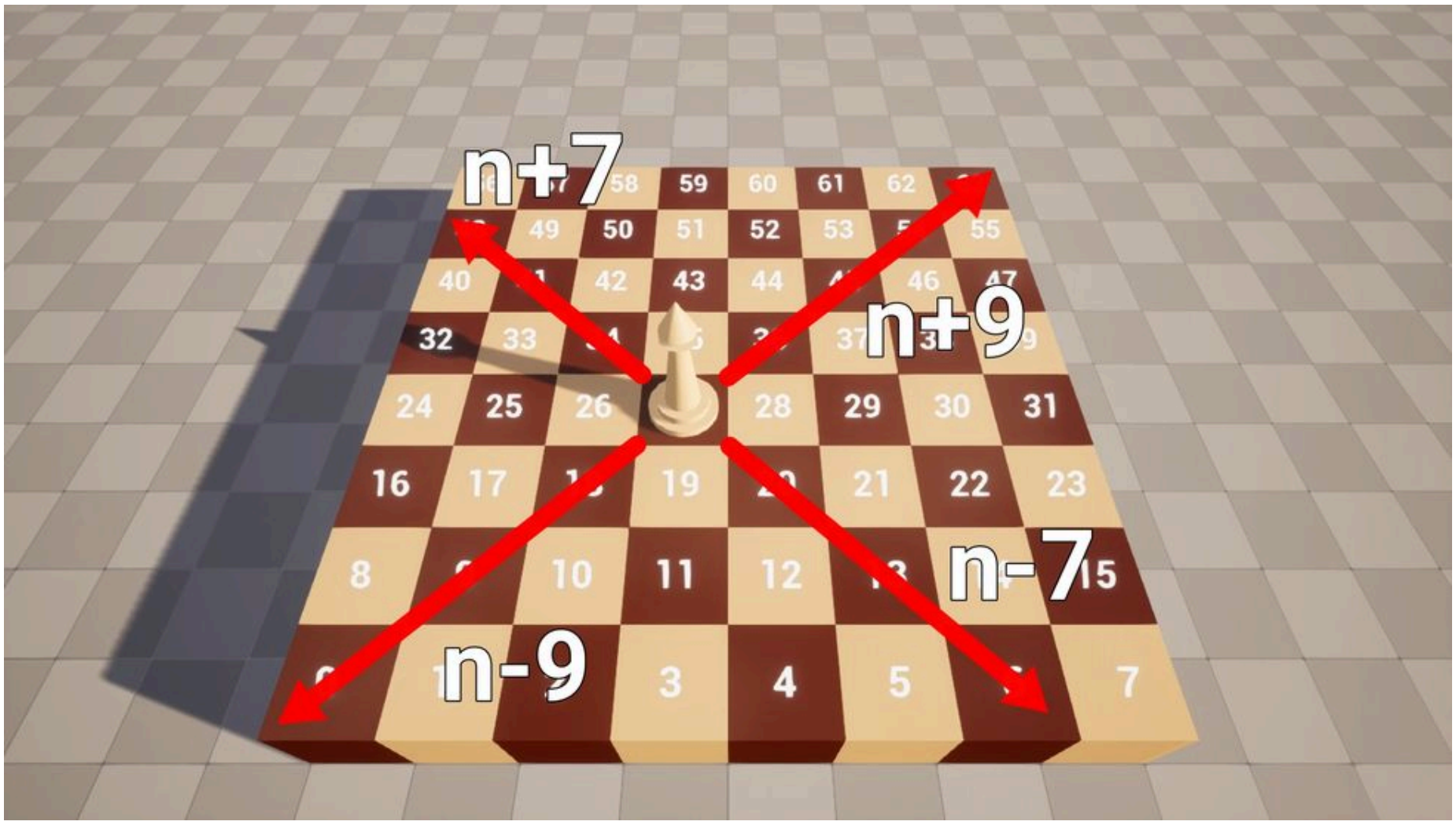


To find what number we need to use for each diagonal, we can use the numbers we used for rook movements. If we move one square backward and then one square to the right, we will find a bottom-right diagonal index. And we know that to move backward, we need to subtract 8 from the index, or we could say that we need to add a -8 to it, and to move right, we add one to it. If we add 1 to -8, we get -7, and that's the number we need to use to find the index values of bottom-right diagonal squares.

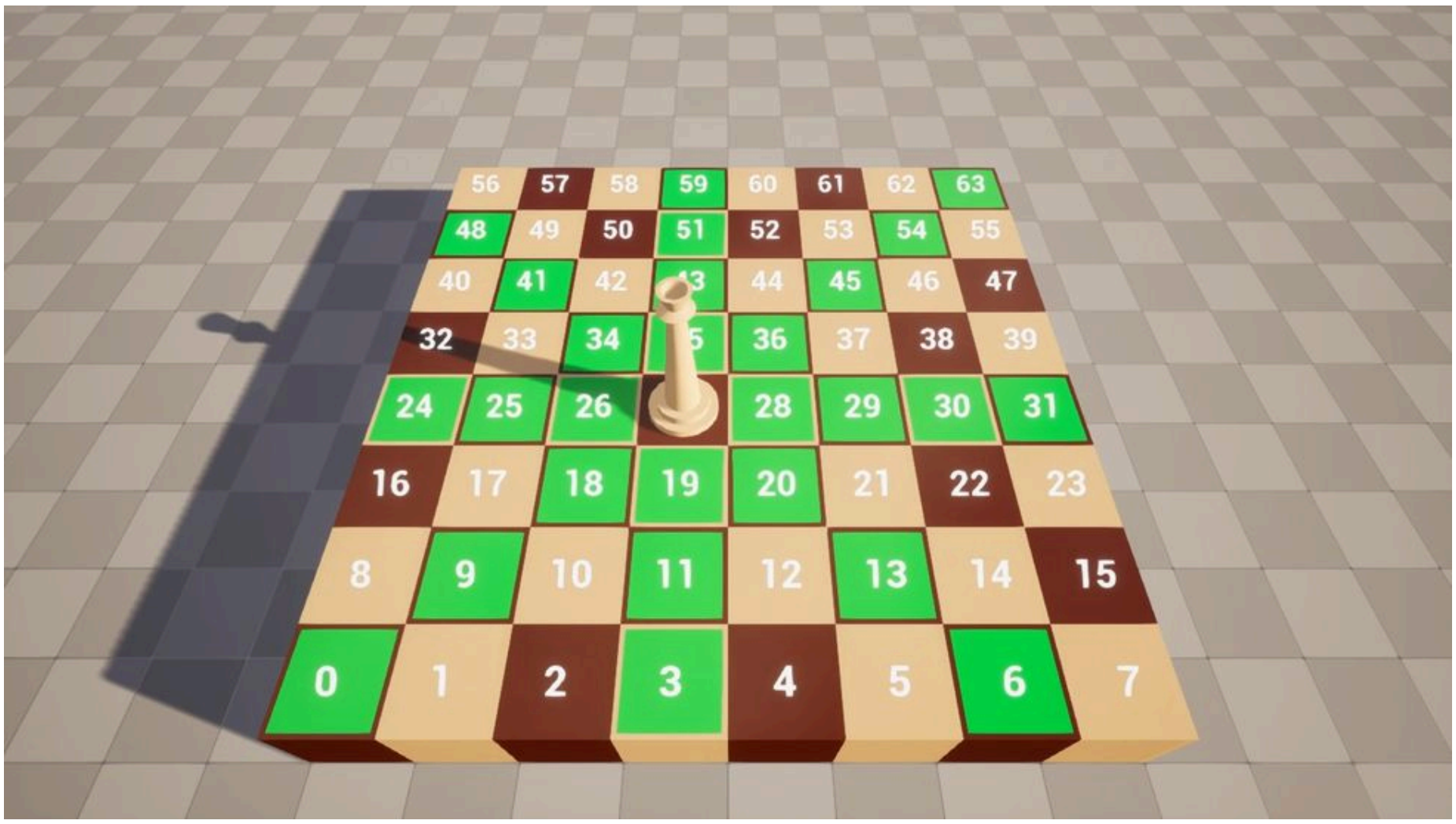


Or we could have taken our bishop index, which is 27, and subtracted it from the bottom square index, which is 20, and we would have found the same value.

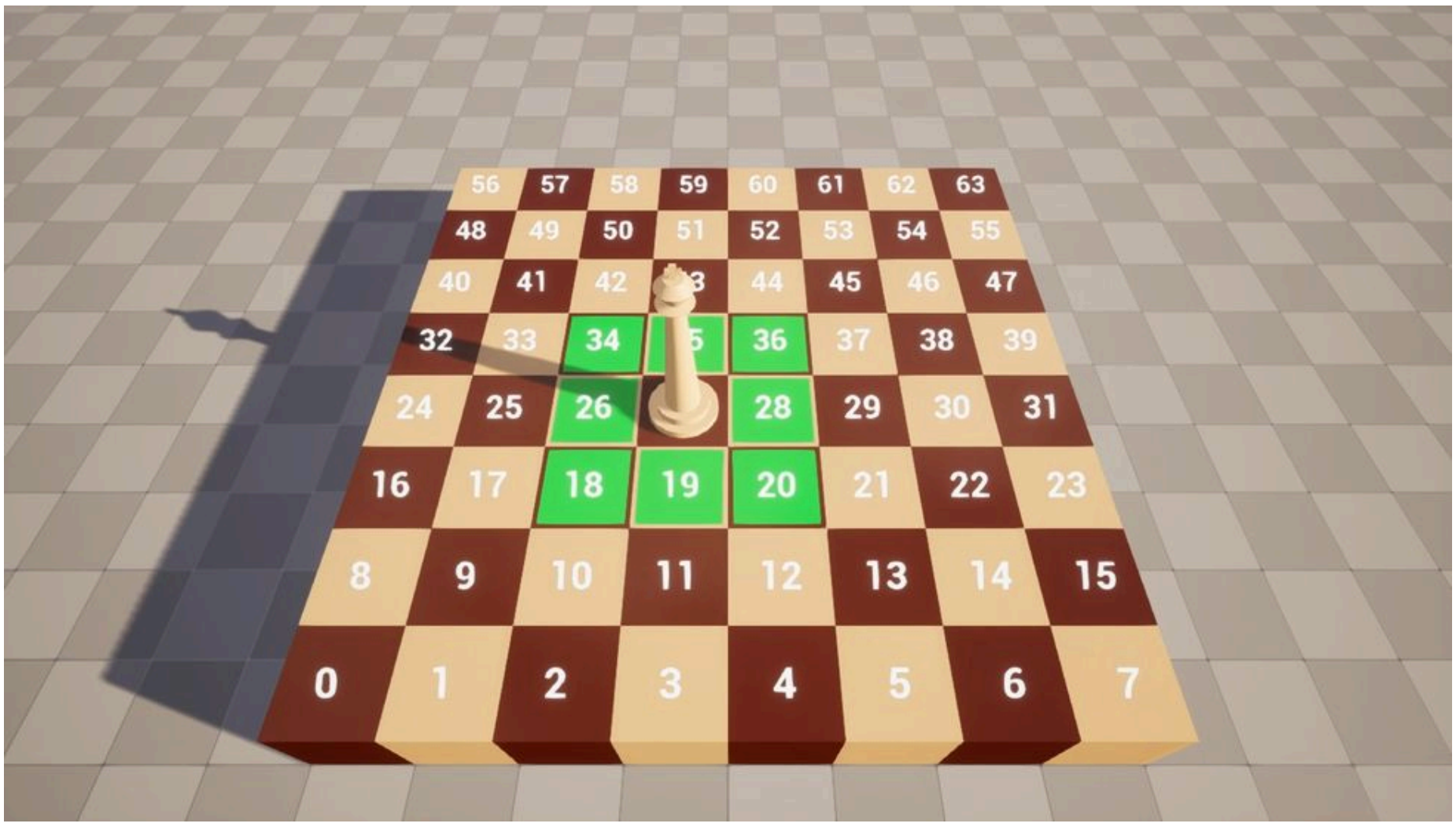
So by using any of those methods, we can find that the other diagonal numbers are -9, 7, and 9. By adding those numbers to index values, we should be able to find all the valid moves for bishops.



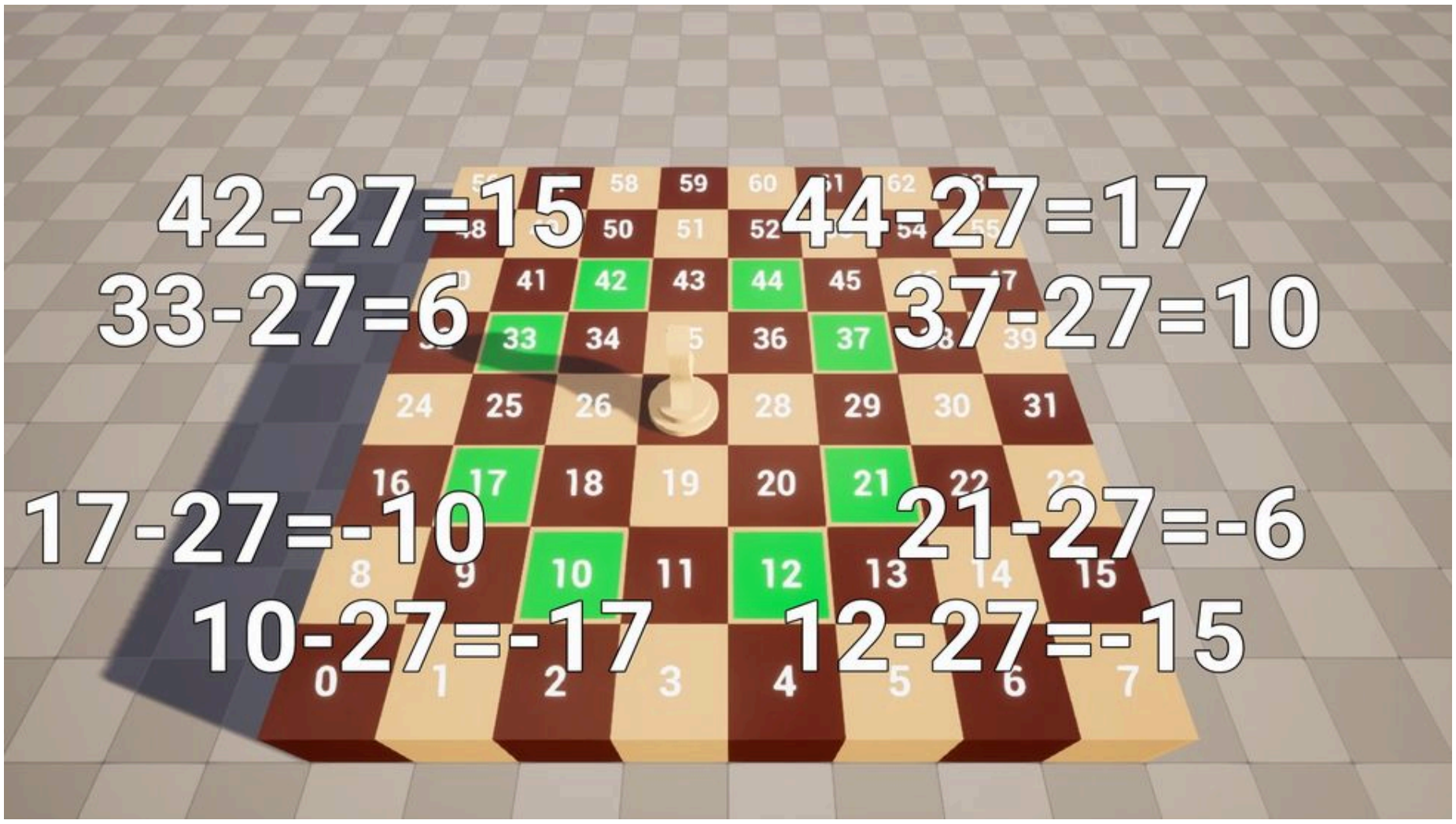
Now that we know how to find moves for the bishops and rooks, we should be able to find all the valid moves for the queen. A queen moves like a rook and bishop combined, so we can simply add rook and bishop movements together and we will get moves for a queen.



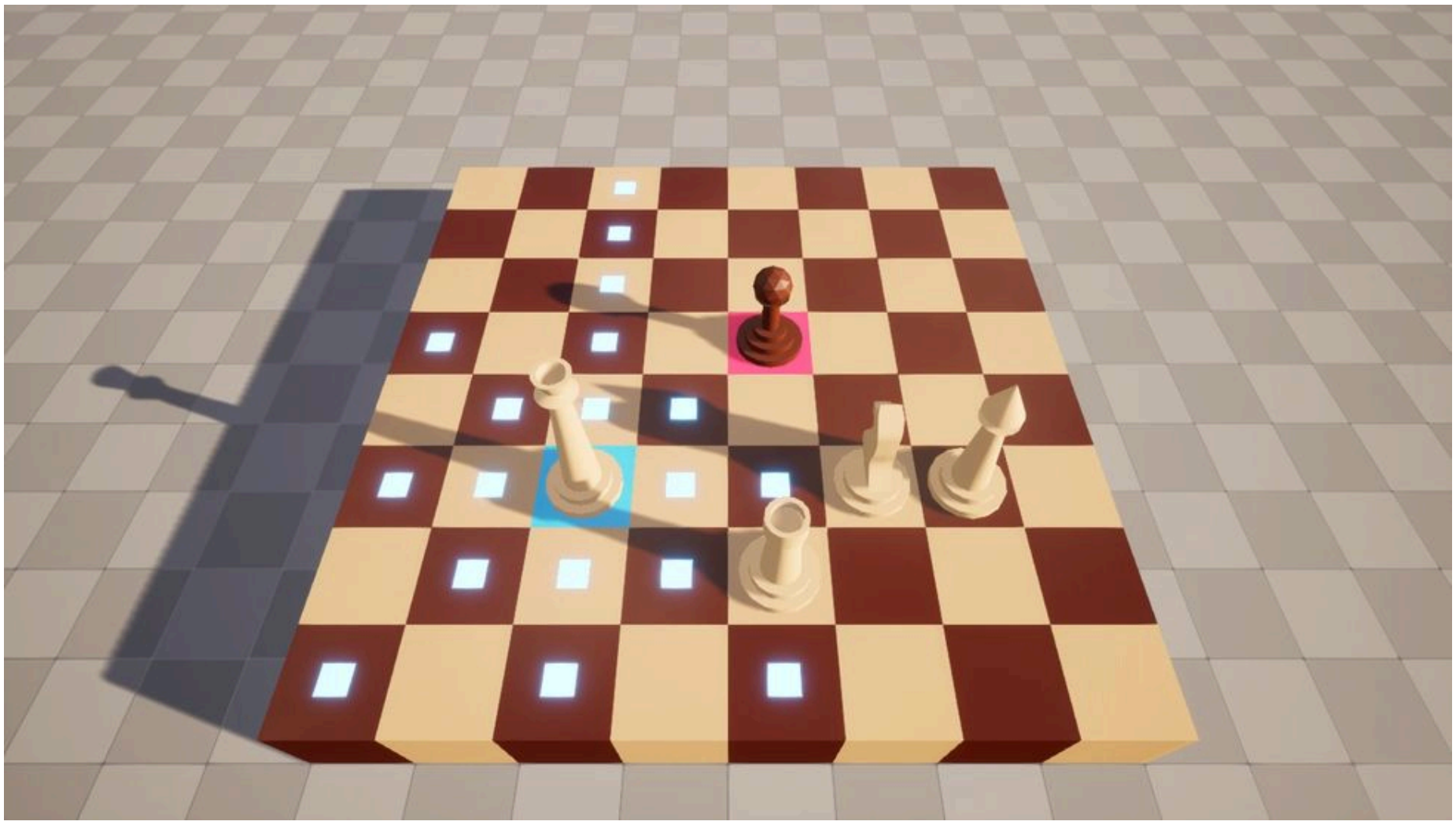
A king can move like a queen in any direction, but it can move only one square, so we just need to look up for one square in each direction.



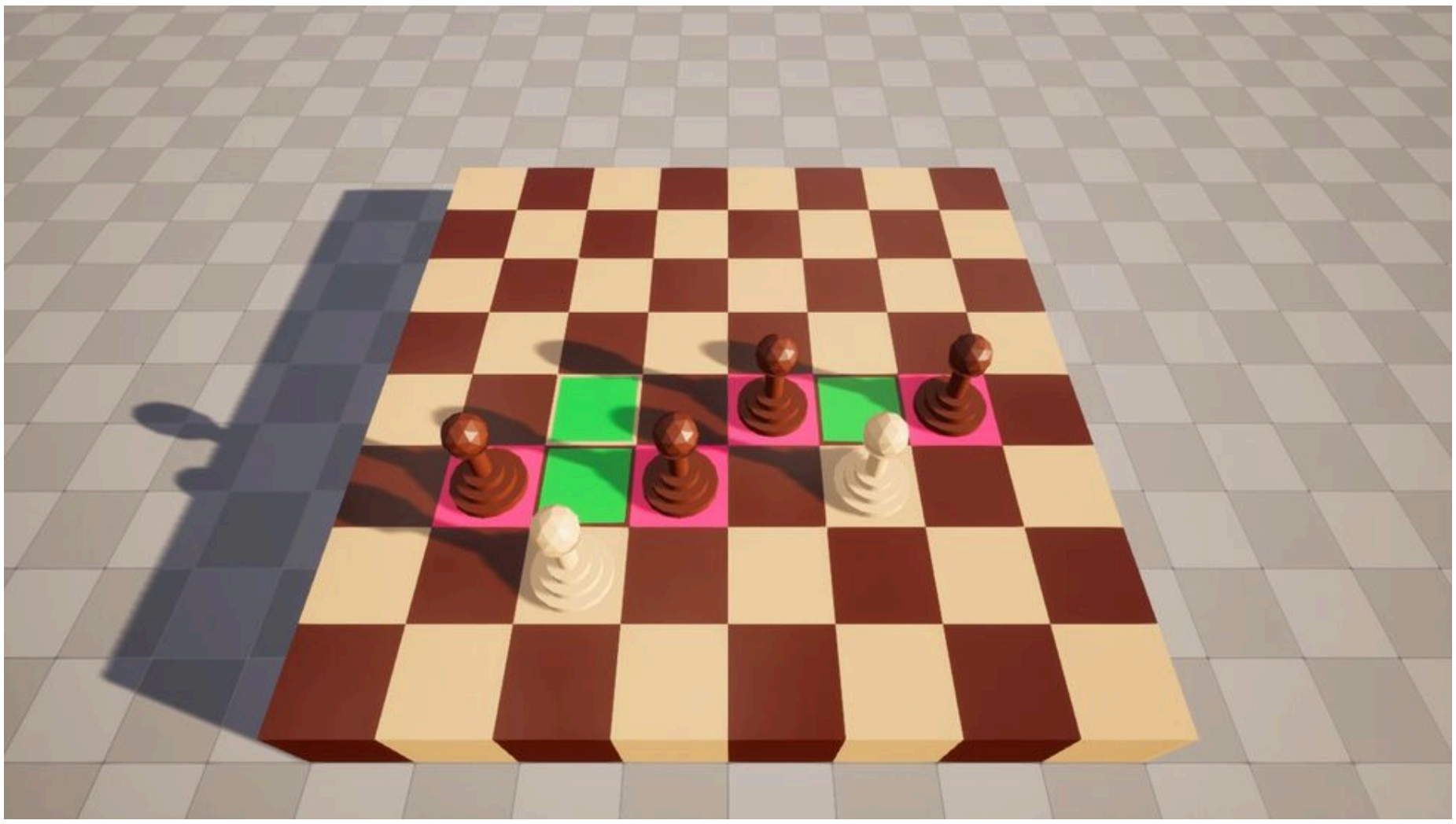
A knight's moves look like capital L letters. So if from each valid move index we subtract the knight piece's index, we will get an offset value for each move. Then, whenever we need to find a knight's valid moves, we can simply add those offset values to the knight piece's index value.



Before looking at the pawns, let's look at how pieces affect each other's movements. If there's an enemy piece on the valid move's square, then that piece becomes capturable, and if that piece is on the path of the queens, rooks, or bishops movement path, then we stop looking for new valid moves further in that direction. If instead of an enemy piece, we have the same color piece, that square becomes an invalid move, and as before, we stop looking for valid moves further in that direction.



If we look at pawns, we will see that their interactions with pieces are different from other pieces. Pawns can move forward one square (two squares if it's the first move for the piece), but in contrast to other pieces, they can't capture enemy pieces while moving forward, any piece invalidates the forward moves for pawns. Pawns can only capture pieces on first-forward diagonal squares, so we can see that pawns have a separate set of moves for capturing enemy pieces and a separate set for regular movement.

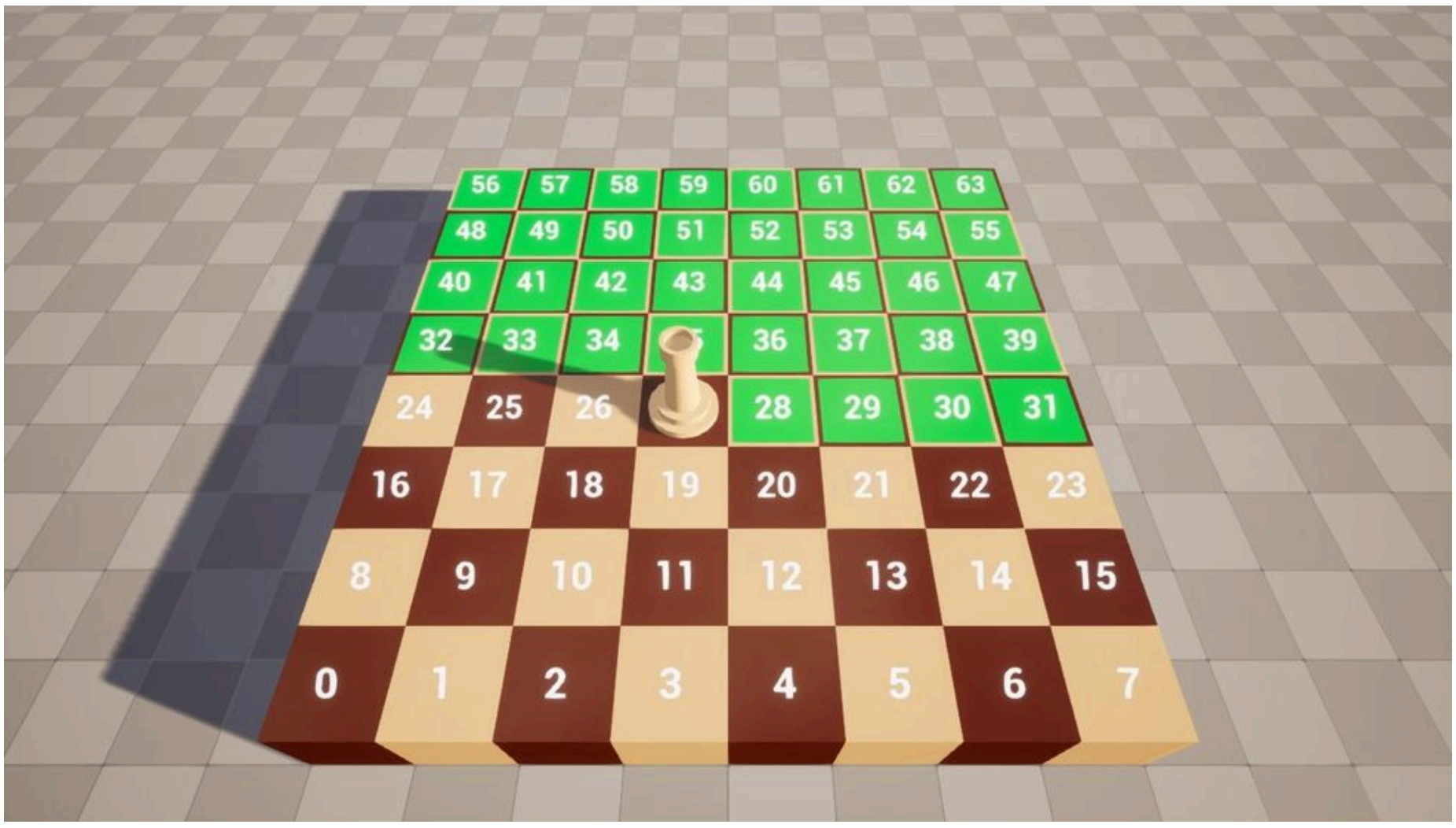


So now I think that I have pretty much figured out all the things I will need for chess logic implementation in my game.

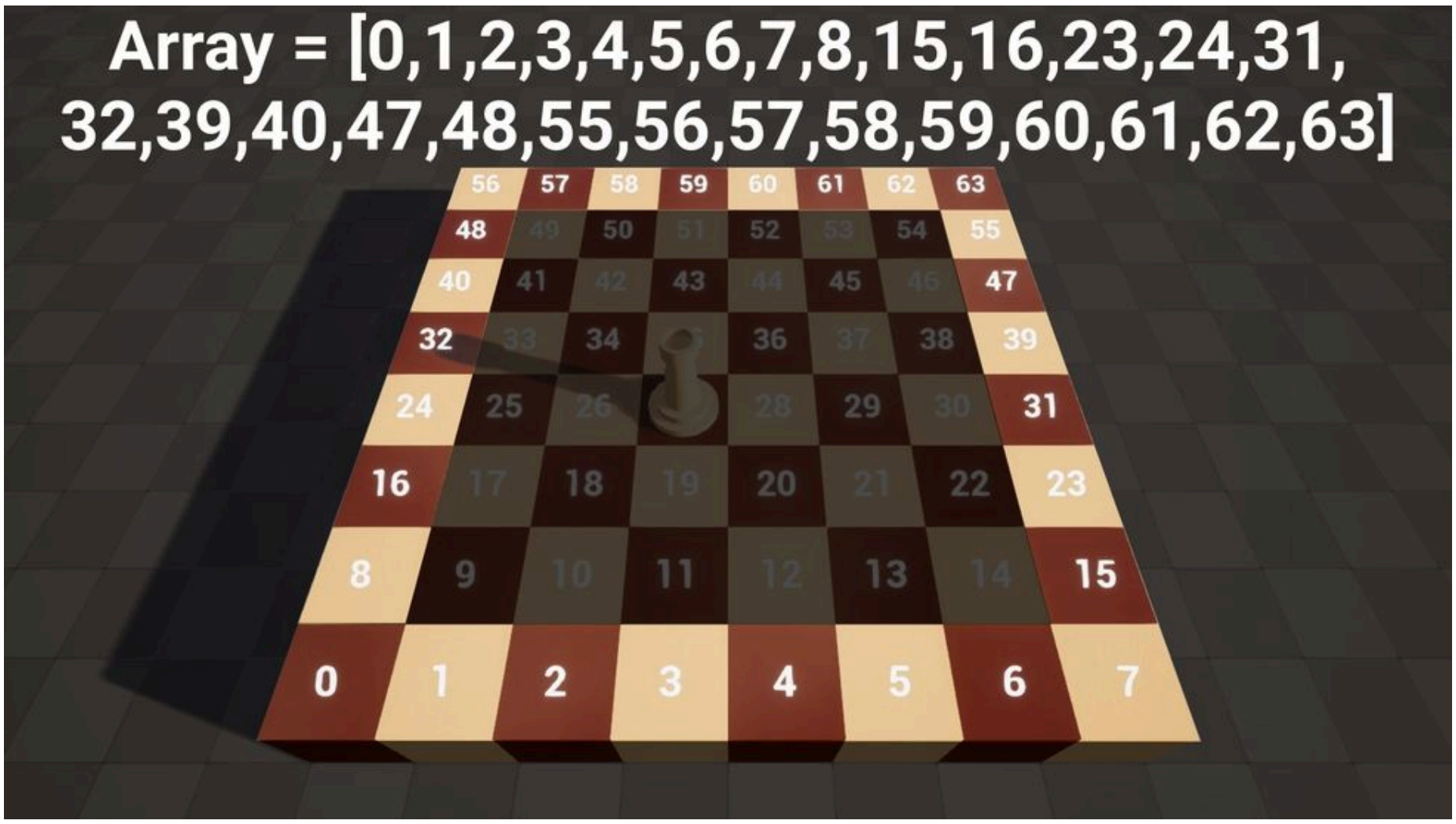
So because I'm using the Unreal Engine to create this game, I can use either blueprints or C++ for logic implementation. Blueprints is a visual scripting system in Unreal Engine that allows you to make games without writing text-based code. more fun than regular C++. So I decided that I would use C++ for most of the logic implementation, because I had a feeling that if I did it in blueprints, it would turn into a big spaghetti mess pretty fast.

After spending way more time than I would have ever expected, I doubtfully think that I may have implemented all the chess logic. I'm not too confident in my implementation, because while doing it, I called myself stupid at least several times more than usual. So when I started to implement the chess logic, I realized that I hadn't figured out how to tell if a square is at the edge of the chessboard.

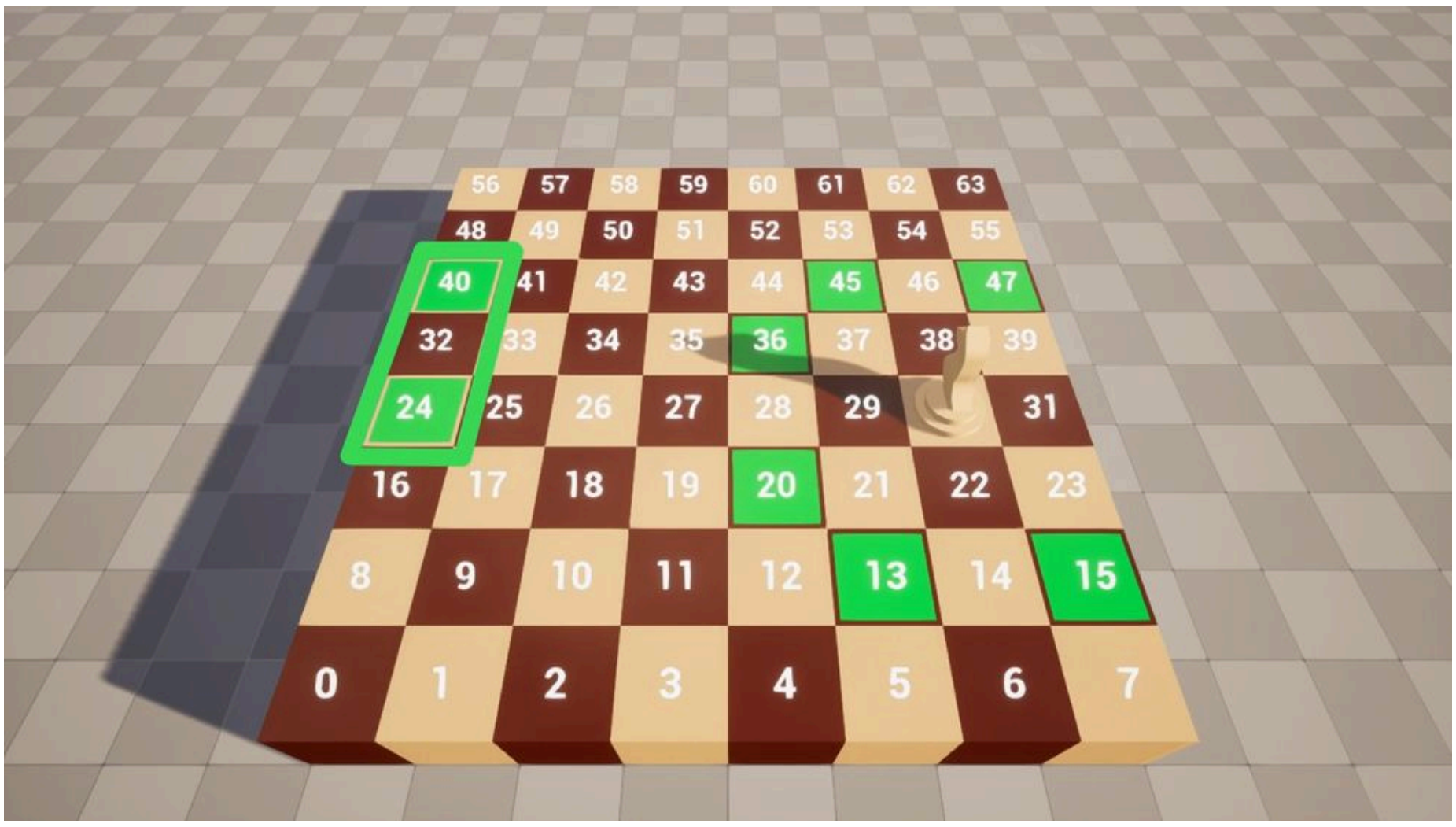
For example, if we want to find all the valid moves for the rook on the right side, we should stop looking for new moves when we reach the edge of a chessboard, but if we don't know when we reach the edge, we will keep setting valid moves for the rook until we run out of squares on the board.



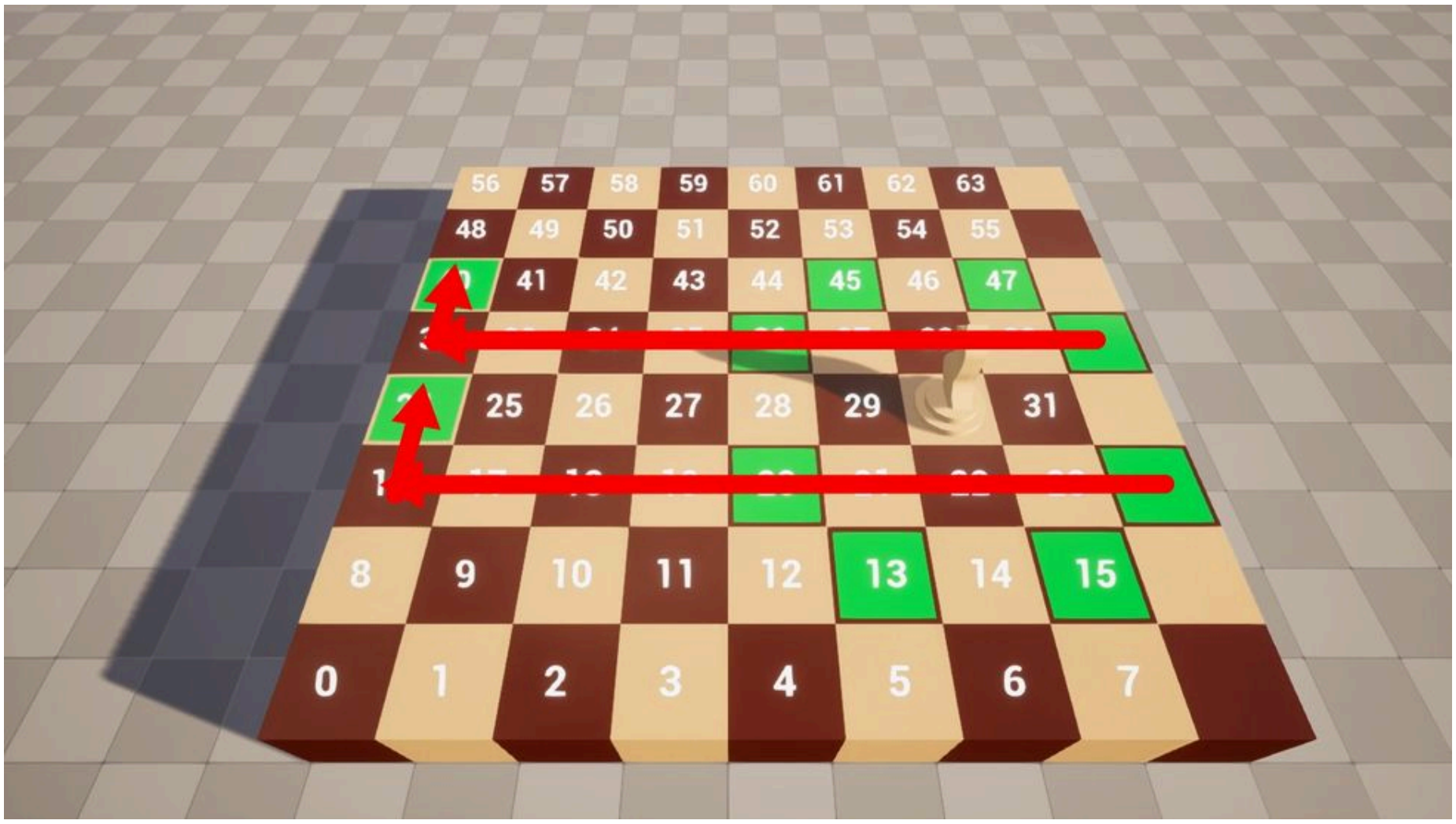
One way to fix this issue would be to make an array and then put all the square indexes that are on the edge of the chessboard in it. Then, to check if a square is on the edge of the chessboard, we could simply check if the square's index is in the array.



This approach would work well enough if we didn't have knight pieces. Because knight pieces jump over squares, there might be situations where a knight piece near the edge would show illegal moves as valid on the other side of the board.



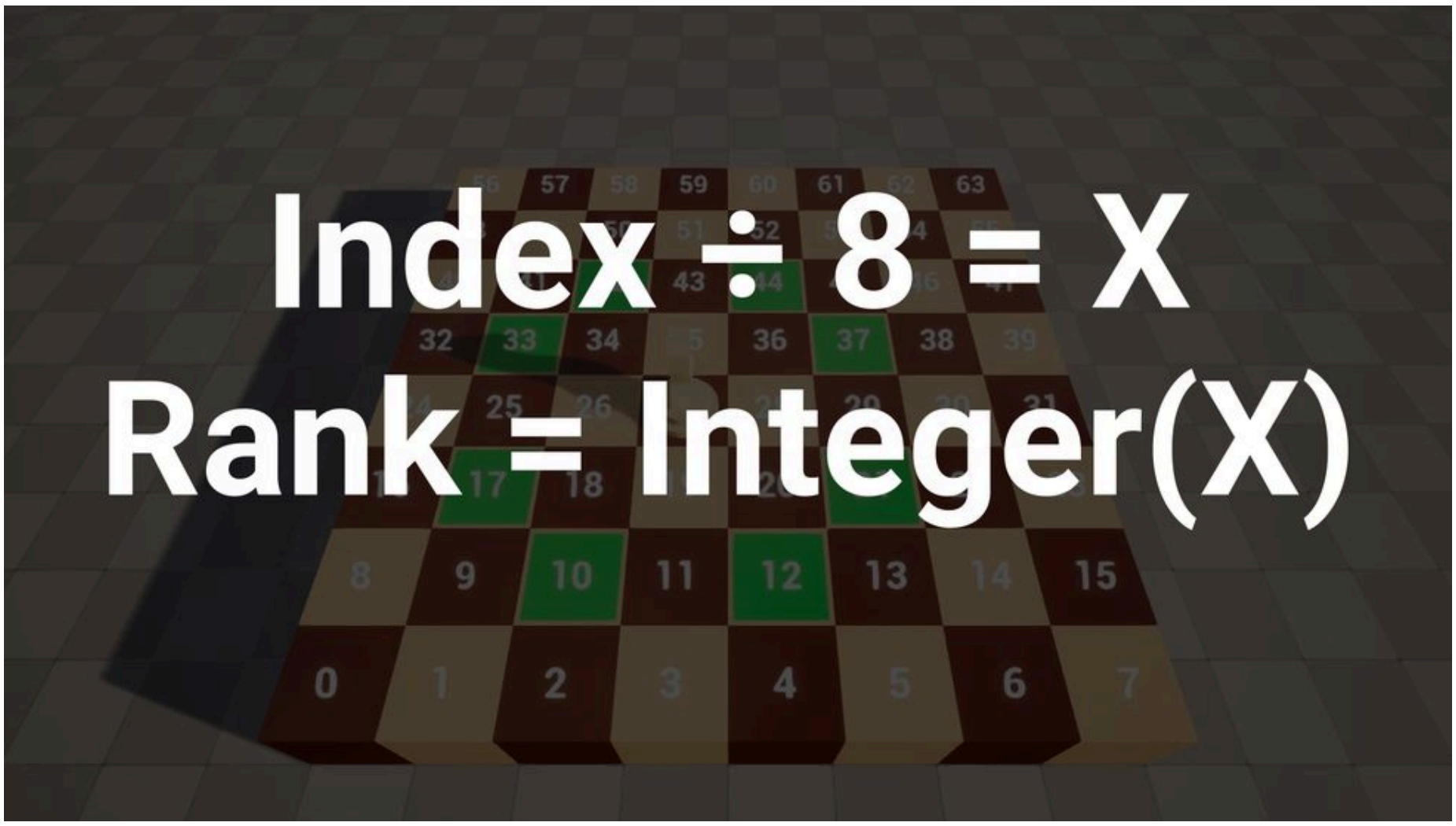
If we extended the board on the right side and placed moves on it that should have been over the edge and compared them to the illegal moves that appeared on the other side of the board, we would notice that illegal moves are shifted by one row. So if we knew on which row the move should be placed, we could simply check if the move would be placed on it and know if it would be a valid move.



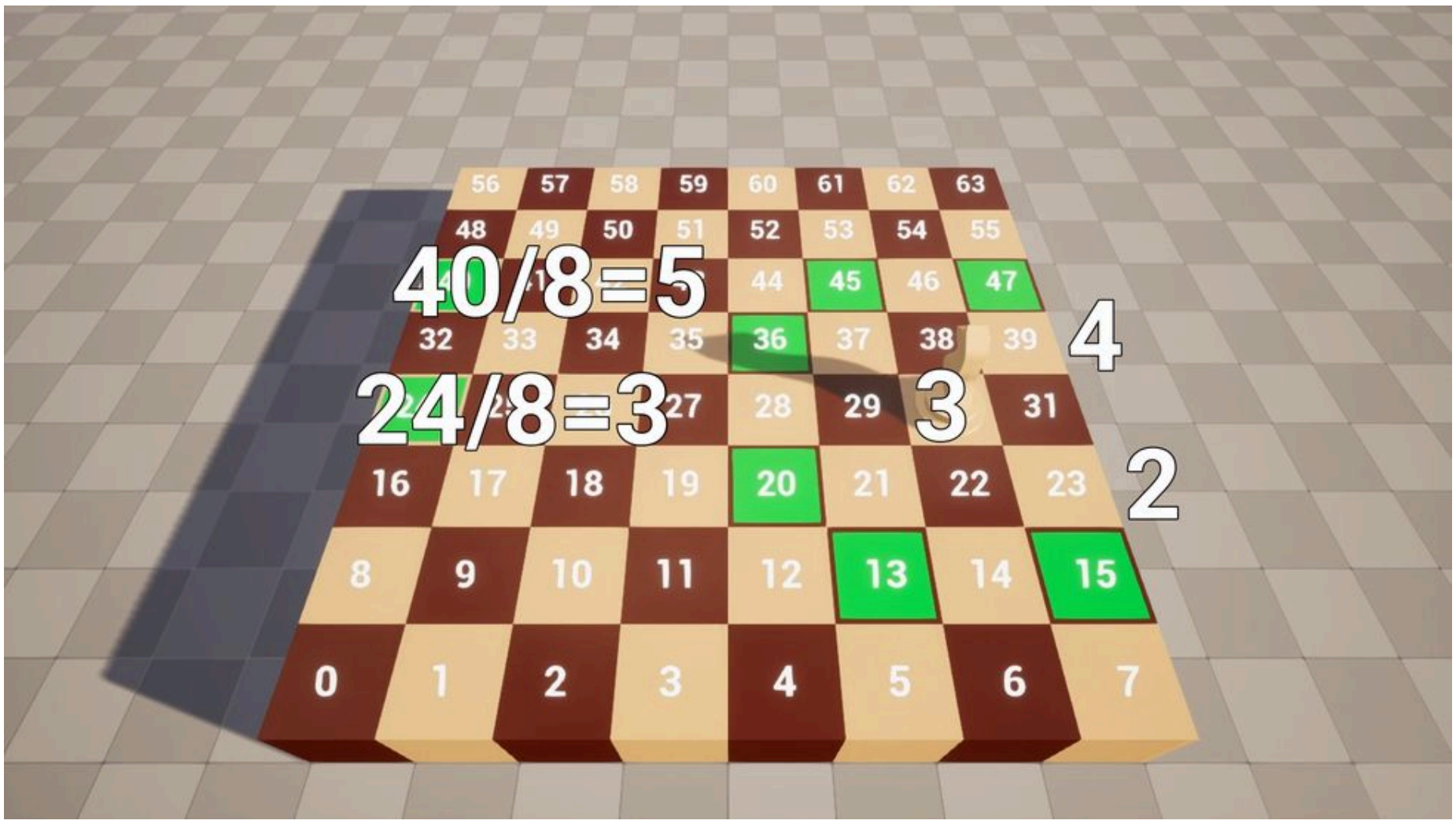
By the way, the columns of a chessboard are known as files, and the rows are known as ranks, so now I will be calling them by those names to look smarter and make things seem more confusing than they are.

So to know on which rank each move should be placed, we can look at the chessboard and simply count how many ranks of the knight piece each move is, and then save that value next to the knight move offset value. But before we can utilize those rank offset values, we need to know how we can get rank values from index values.

To get the rank value of the index, we need to divide it by eight, which is the width of the chessboard, and then the integer value of the division is the rank value of the index.

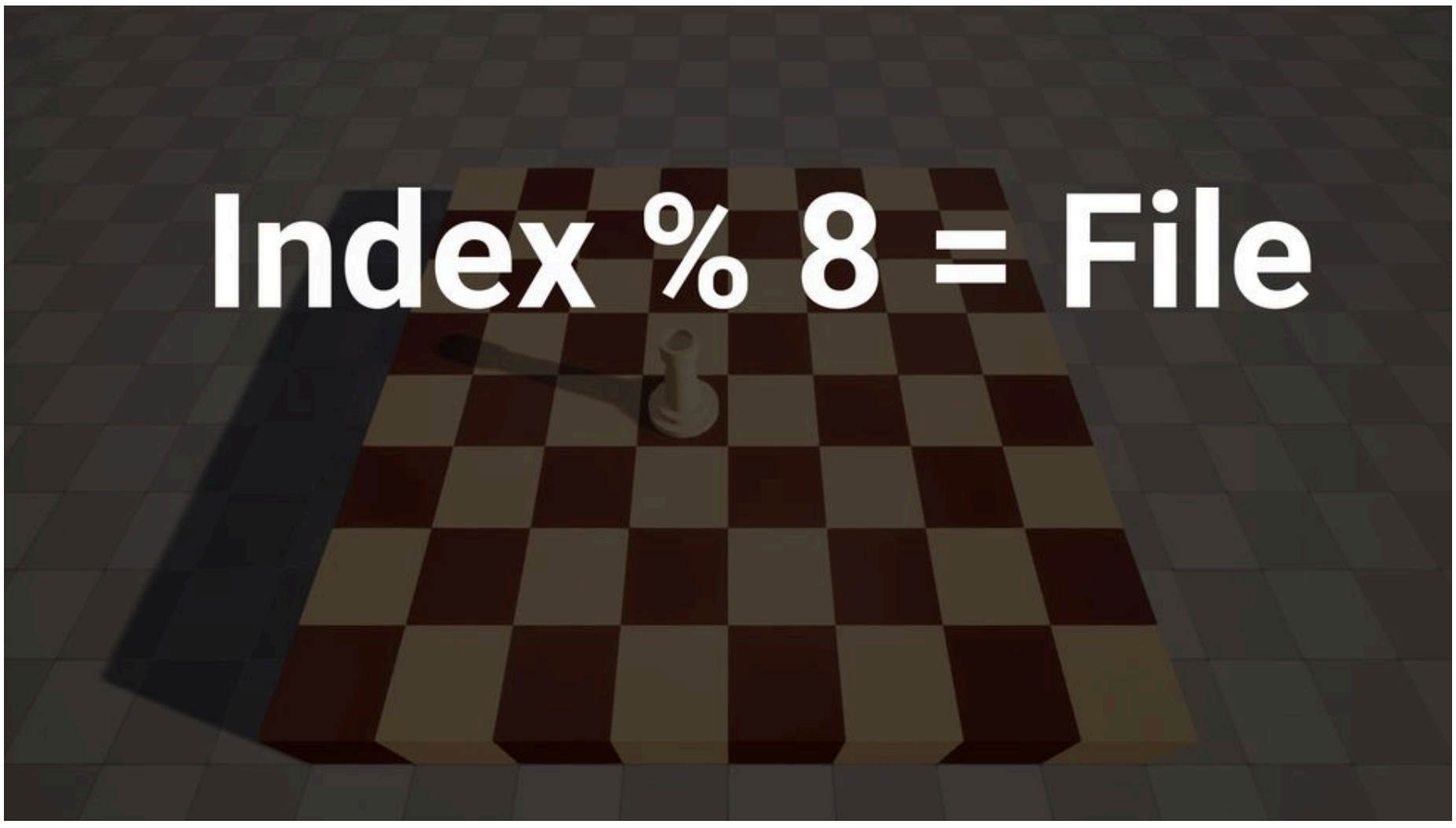


So if we look back at the previous position, we can see that the knight piece rank value is 3. And the furthest right-move rank offset values are -1 and 1. So we know that moves should be placed in ranks 2 and 4. We can see that the move index values are 24 and 40, and by calculating ranks for them, we get 3 and 5, which doesn't match our expected rank values, so we now know that those moves are not valid.



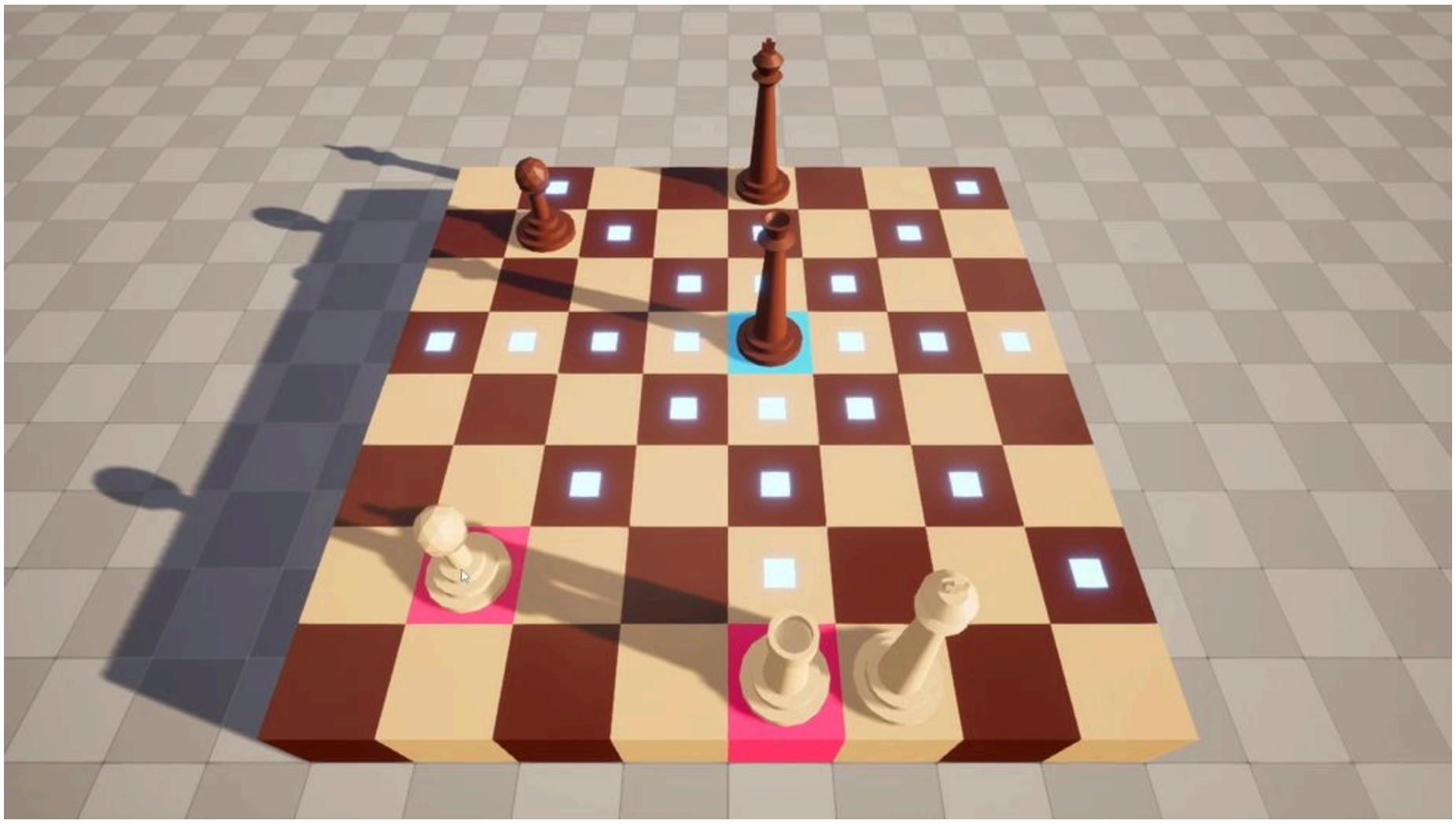
We can also use rank values to find out if a square is at the bottom or top edge of the board. If the rank value is 0 or 7, then we know that the square is at the bottom or top edge of the chessboard. We can also do the same with the file values If the square file value is 0 or 7, then it's at the left or right edge of the board. But to utilize this method to find the chessboard edges, we need to figure out how to get file values from index values.

To get the file value of the index, we need to divide it by eight, which is the height of the chessboard, and then the remainder of the division is the file value of the index. To get a remainder from a division, we can use the modulo operator, which usually in programming languages looks like a % sign. To use it, replace the division sign with the modulo operator, and you will get the remainder of the division.

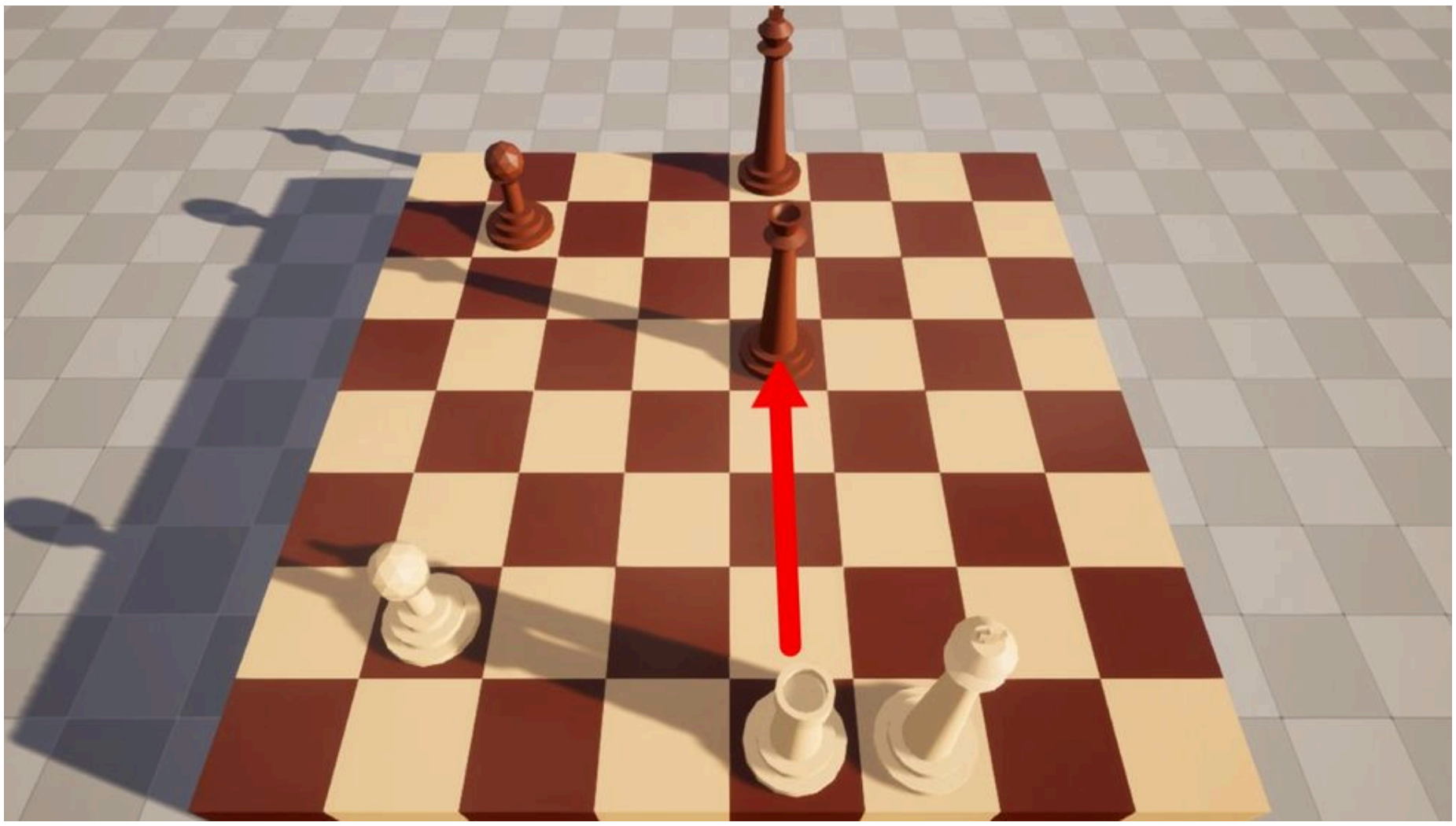


After figuring these things out, I could implement each piece's movement logic, but it turns out that you need more than that to make a fully functional chess game.

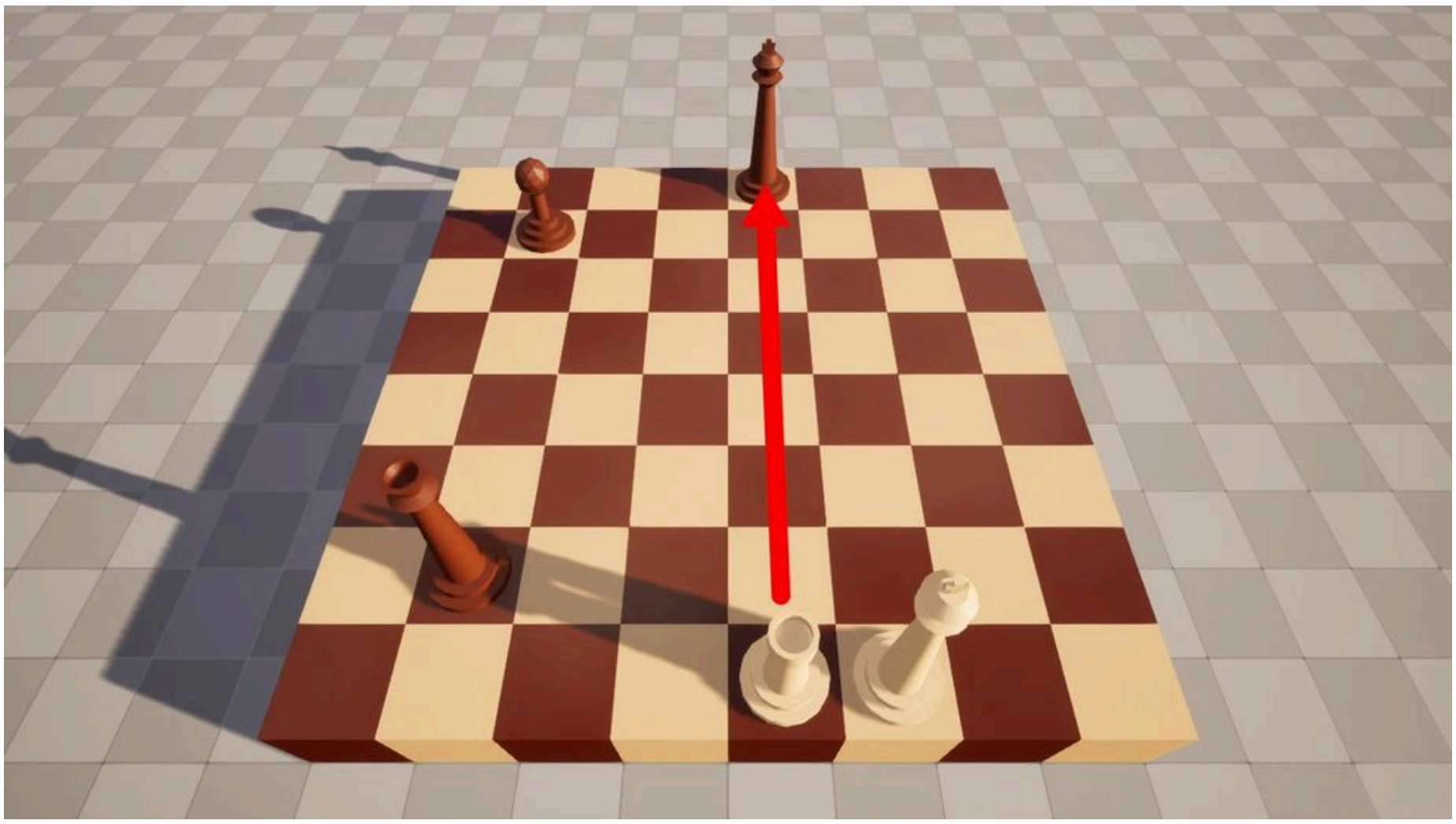
Let's have a look at this position on the board.



If we look at the valid moves of the queen on the board, they don't seem wrong, and this move also looks totally valid. But if we look a little bit closer, we will see that a queen is pinned by the white rook.



And if we move the queen out of its current file, then the black king piece will be checked by the white rook piece, and that's not a valid move. So I also had to figure out how to find pinned pieces and make them show only valid moves.

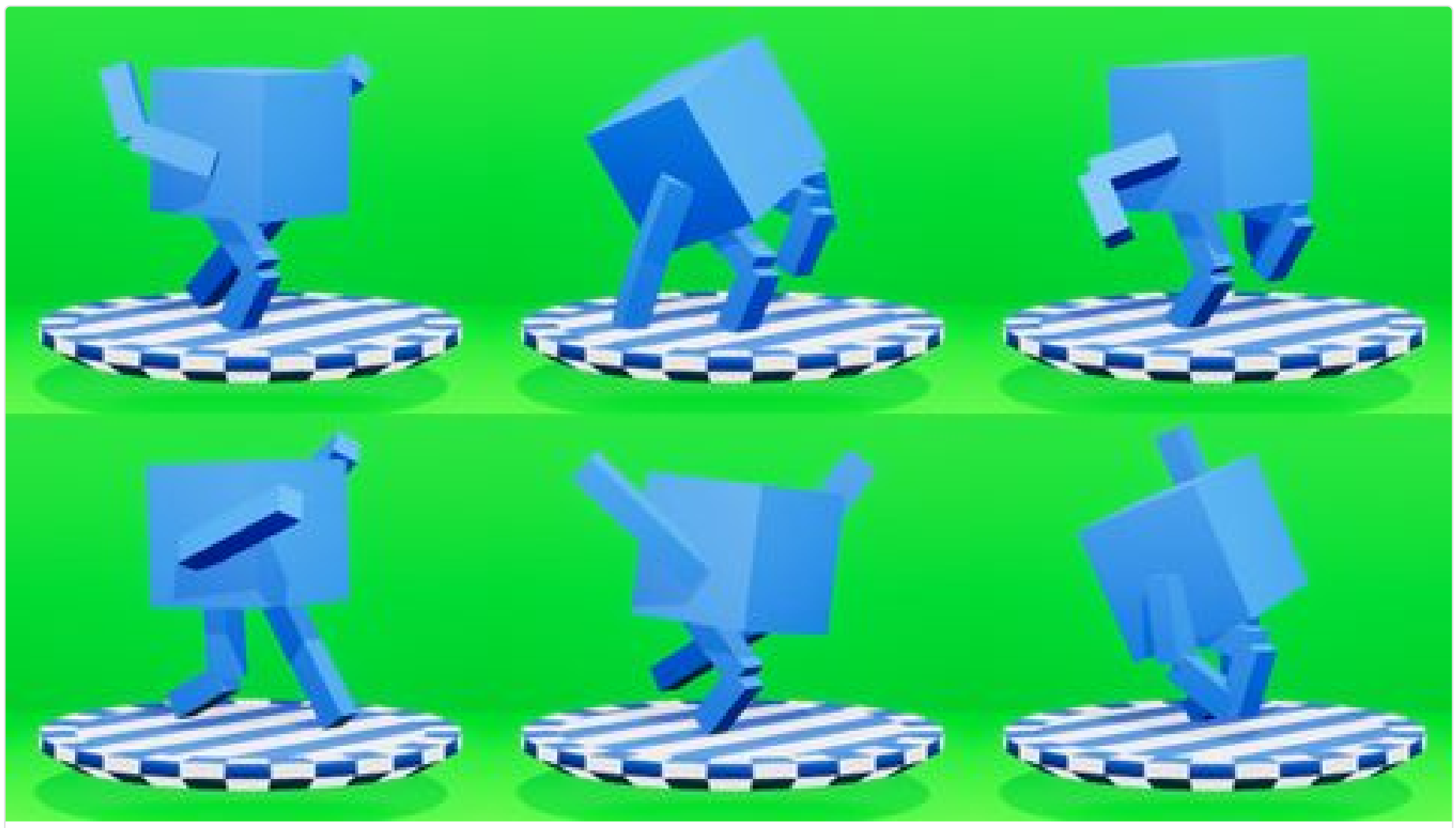


And another thing I had to figure out was how to tell when it's check, checkmate, or one of the draw states. Also, there's this rule called en passant. And I also managed to implement castling. And let's not forget about pawn promotions.

So now I hope that after writing all this bad code, I managed to make a fully functional chess game. And now I probably could start to implement gacha mechanics, but there is one issue that's preventing me from doing that. There's nobody to play against now, and I doubt that I would get way better at chess if I played only against myself. So there are two ways I could get opponents for this game. I could make it into an online multiplayer game, but it's probably lots of additional work, and there's no way that it will be played by enough people that you could find an opponent at any time of the day. Or I could make some chess AI to be my opponent, so then there would be an opponent to play against all the time. And I think that implementing chess AI is the best choice, but I think that I'll leave that for another devlog.

Share on

Related Articles



Procedural walking animation generator in Unreal Engine 5